CASE FILE

SOLSTICE BREACH

A Seven-Stage Digital Forensics & Penetration Testing Investigation

CASE NO. SDX-2026-0914 · CLASSIFICATION: RESTRICTED · STATUS: FOR TRAINING DISTRIBUTION

Document	Solstice CTF — Full Case File, Architecture & Deployment Plan
Revision	v3 — Added managed-platform hosting (Render/Railway/Fly.io) and a Vercel compatibility note to the Deployment Plan
Scope	All 7 stages (Exhibits A, B, C, D, D½, E, F) — scenario, tasks, solve method, architecture, deployment
Audience	CTF maintainers / instructors — this is a spoiler document, not a player handout
Redaction policy	All flags, passwords, access codes and final discovered answers are redacted (black bar) inline, immediately after each task. Text is present but visually hidden — select/copy in a PDF viewer to reveal.
Application version	Post-restructure layout — code under `src/`, runtime data under `data/`, frontend under `public/`, docs under `docs/`, hosting configs under `deploy/`

Generated for internal use. Nothing in this application touches a real filesystem, credential, or network.

Table of Contents

1. **Part I — Program Overview**
    a. **Concept & Narrative**
    b. **Learning Objectives**
    c. **Full System Architecture**
    d. **Repository & File Structure**
    e. **Session Model (Two Independent Layers)**
    f. **PeopleHub Database Schema**
    g. **Role-Based Access Control Matrix**
    h. **Progress / Flag State Machine**
2. **Part II — The Seven Stages**
    1. **Stage 1 — Exhibit A: Open-Source Intelligence**
    2. **Stage 2 — Exhibit B: EXIF Metadata Forensics**
    3. **Stage 3 — Exhibit C: The Solstice PeopleHub Investigation**
    4. **Stage 4 — Exhibit D: XXE Injection & IDOR**
    5. **Stage 5 — Exhibit D½: The Turn (narrative checkpoint)**
    6. **Stage 6 — Exhibit E: Apache Log Analysis**
    7. **Stage 7 — Exhibit F: Disk Forensics & Case Closed**
3. **Part III — Deployment Plan (Internet Hosting)**
    a. **Hosting Model — What "On The Internet" Means Here**
    b. **Recommended Topology (Diagram)**
    c. **VPS Provider Comparison & Sizing**
    d. **Step-By-Step VPS Deployment Runbook**
    e. **Managed Platform Hosting — Render, Railway, Fly.io**
    f. **Why Not Vercel**
    g. **Environment & Secrets Handling**
    h. **Reset / Reseed Operational Runbook**
    i. **Backup & Recovery**
    j. **Security Hardening Checklist**
4. **Appendix — Environment Variable & File Map Reference**

Part I — Program Overview

1.1 Concept & Narrative

Solstice CTF is a fully local, dependency-free web application built to teach practical application-security skills through a single continuous investigation rather than isolated puzzles. The player is cast as an incident responder handed a breach case file for the fictional company **Solstice Dynamics Ltd.** Each of the seven stages hands over one piece of evidence and one class of real vulnerability; solving a stage produces a flag, and that flag is the key that unlocks the next stage. The centerpiece, Stage 3, drops the player into **Solstice PeopleHub** — a complete, realistic fictional HR platform (~1,000 synthetic employees, 25 database tables, 9 roles, 23 application pages) that must be assessed the way a real internal tool would be: nothing in the interface points at where the bugs are.

Everything in the application is synthetic — fictional company, fictional people, fictional documents, and a fictional sandboxed filesystem for the XXE lab. No stage ever touches a real filesystem path, a real credential, a real network, or sends data anywhere outside the machine it runs on.

1.2 Learning Objectives

Vulnerability class	OWASP mapping	Where it is taught
OSINT / metadata analysis	A05/A09 — adjacent skill, not a CWE	Stage 1, Stage 2
SQL injection — auth bypass	A03:2021 Injection	Stage 3 (PeopleHub login)
SQL injection — UNION-based data exfiltration	A03:2021 Injection	Stage 3 (Directory, Profile, Reports, HR Requests)
Broken object-level authorization (IDOR)	A01:2021 Broken Access Control	Stage 3 (Documents), Stage 4 (Payslips)
XML External Entity (XXE) injection	A05:2021 Security Misconfiguration / A03 Injection	Stage 3 (Employee Data Import), Stage 4
Stored Cross-Site Scripting (XSS)	A03:2021 Injection	Stage 3 (HR Feedback)
Business-logic / break-glass bypass	A04:2021 Insecure Design	Stage 3 (Emergency Access)
Log analysis / attacker attribution	Detection & response skill	Stage 6
Disk forensics (thumbnail cache, Prefetch)	Detection & response skill	Stage 7

1.3 Full System Architecture

The application is deliberately dependency-free: one vanilla Node `http.createServer` (`src/server.js`), `node:sqlite` for persistence, and hand-written multi-page HTML/CSS/JS on the frontend. There is no build step and nothing is installed from npm.

```
Browser
  |
  v
src/server.js  (single vanilla http.createServer, no framework)
  |
  |-- /api/login, /api/resume, /api/progress, /api/exhibit-*  --> outer game (stages 1,2,4,5,6,7)
  |
  |-- /api/peoplehub/*   --> requires outer session, stage3 = true
  |                           --> src/lib/peoplehub/router.js (handle)
  |                                --> requires PeopleHub's OWN session (phsid cookie)
  |                                --> routes/*.js handlers, RBAC-checked (rbac.js)
  |                                --> src/lib/peoplehub/db.js (one persistent sqlite connection)
  |
  |-- /api/ctf-admin/*   --> requires loopback connection only (no session of any kind)
  |                           --> src/lib/peoplehub/router.js (handleAdmin)
  |
  \-- everything else (incl. all /peoplehub/*.html) --> static file serving (public/)
```

1.4 Repository & File Structure

The project is organized around a strict separation of *code*, *generated data*, *static frontend*, *documentation* and *hosting configuration*:

```
thahani/
|-- src/                    ALL backend code (nothing generated lives here)
|   |-- server.js           outer game + PeopleHub mount points
|   |-- lib/
|   |   |-- normalize.js, forensics.js, hrdb.js, xxe.js     outer-game engines
```

```
|   |   \-- peoplehub/
|   |       |-- db.js            persistent sqlite connection
|   |       |-- session.js       PeopleHub login sessions (phsid)
|   |       |-- rbac.js          9-role capability matrix
|   |       |-- challengeState.js  9-flag progress tracker (keyed by OUTER session token)
|   |       |-- router.js        dispatcher + handleAdmin (dev reset panel)
|   |       \-- routes/*.js      one file per business area (12 files)
|   \-- seed/
|       |-- config.js, data/, lib/rng.js, generators/*.js
|       |-- seed.js              rebuilds data/db/peoplehub.sqlite from scratch
|       \-- outer-seed.js        rebuilds data/users.json + data/challenges.json
|
|-- public/                  static frontend — no build step
|   |-- index.html, map.html, exhibit-*.html, gate-*.html, stage2.html, epilogue.html
|   \-- peoplehub/            23 pages, shared/ (api.js, shell.js, shell.css, table.js)
|
|-- data/                    everything generated or read at runtime (safe to delete + rebuild)
|   |-- db/schema.sql, db/peoplehub.sqlite
|   |-- challenge-data/       XXE lab's synthetic sandboxed filesystem
|   |-- evidence/             outer game's downloadable evidence files
|   \-- users.json, challenges.json
|
|-- deploy/                  hosting configuration
|   |-- Caddyfile             reverse proxy + admin-panel path blocking (VPS path)
|   |-- solstice-ctf.service  systemd unit for process management (VPS path)
|   \-- render-start.sh       conditional seed-then-start script (Render path)
|
|-- docs/                    CHALLENGE_GUIDE.md, this case file, .docx exports
|-- tools/                   cloudflared.exe
|-- package.json, Dockerfile, docker-compose.yml, .gitignore, .dockerignore
```

1.5 Session Model — Two Independent Layers

Layer	Cookie	Storage	Governs
Outer CTF session	`sid`	In-memory `Map` in `src/server.js`	Story progression: `stage1..stage6`, `solved1..solved6`. PeopleHub is completely unreachable without `stage3 = true` here.
PeopleHub session	`phsid`	In-memory `Map` in `src/lib/peoplehub/session.js`	"This browser is signed into the HR platform" — `userId`, `employeeId`, `roleId`, `roleName`. Knows nothing about challenge progress.

Both session layers are plain in-memory Maps inside one Node process — the whole reason this app cannot run as short-lived serverless functions (see Part III §3.6). Challenge progress for Stage 3's nine sub-flags lives in the `challenge_state` table inside the PeopleHub *database* instead — not in either session object — keyed by the **outer** session's token, so a player can log out and back into PeopleHub, or even escalate from one PeopleHub account to another, without losing progress within the same outer-game run.

1.6 PeopleHub Database Schema

25 tables, built once by `node src/seed/seed.js` into `data/db/peoplehub.sqlite` and never regenerated per-request:

```
departments --+-- employees >---- manager_id (self-referential)
              |        |
              |        +--< employee_profiles (1:1)
              |        +--< payroll (1:1)
              |        +--< leave_records
              |        +--< performance_reviews (also reviewer_id -> employees)
              |        +--< documents (owner, nullable) --< document_access >-- roles
              |        +--< hr_requests / it_tickets (also assigned_to -> employees)
              |        +--< training_enrollments >-- training_courses
              |        +--< notifications
              |        +--< security_events
              |        +--< emergency_access_requests (requested_by + 3 approver FKs -> employees)
              |        \--< interviews (interviewer_id -> employees)
              |
              \--< job_openings --< applications >-- candidates
                                          \--< interviews

users --> roles (role_id), employees (employee_id, nullable)
audit_logs --> users (user_id, nullable -- system actions)
reports --< report_results
```

```
import_jobs                     (no FK -- standalone log)
challenge_state                 (session_key = outer game's session token, not an FK)
```

1.7 Role-Based Access Control Matrix

Role	Directory	Any Profile	Team Profiles	Payroll	Reports	HR Admin	IT Admin	Recruitment	Audit	Security	Admin	Emergency Approve
Employee												
Manager	✓		✓									✓
HR Helpdesk	✓	✓										
HR Manager	✓	✓	✓	✓	✓	✓		✓				✓
Recruiter	✓							✓				
Payroll	✓			✓	✓							
IT Support	✓						✓					
Security	✓	✓			✓				✓	✓		✓
Administrator	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

Enforced server-side in every route handler (`src/lib/peoplehub/rbac.js`) — the sidebar navigation only hides links a role cannot use; it is never the access-control mechanism itself.

1.8 Progress / Flag State Machine

Every stage follows the same pattern: unlock with the previous stage's flag → solve the puzzle → receive the next flag.

```
Login --> accept case --> STAGE 1 (Exhibit A) --> flag1
  flag1 unlocks --> STAGE 2 (Exhibit B) --> flag2
  flag2 unlocks --> STAGE 3 (Exhibit C / PeopleHub) --> flag3   [requires 9 sub-flags + emergency code]
  flag3 unlocks --> STAGE 4 (Exhibit D) --> flag4
  flag4 unlocks --> STAGE 5 (Exhibit D 1/2, narrative only, no puzzle) --> proceeds automatically
             --> STAGE 6 (Exhibit E) --> flag5
  flag5 unlocks --> STAGE 7 (Exhibit F) --> flag6 --> EPILOGUE / CASE CLOSED
```

STAGE 1 / 7

Exhibit A — Open-Source Intelligence

EVIDENCE: PHOTOGRAPH + METADATA FRAGMENTS

Scenario

The player receives a small bundle of open-source clues tied to the first sighting of the intruder: a street photo, a café, a phone number fragment, and a short video. The task is pure OSINT — reverse-image and reverse-lookup work outside the application itself.

Tasks

- Identify the city and exact street shown in the photo.
- Identify the café visible in the background.
- Recover the phone number associated with the location.
- Identify the video referenced by the evidence (by link, bare video ID, or exact title).

How To Solve

Reverse-image-search the provided photo; cross-reference visible signage and storefronts against maps of the identified neighborhood; use the café's public listing to recover the associated contact number; match the video clue against its platform listing by title or thumbnail. All four answers are normalized server-side (case, spacing and punctuation-insensitive) before being checked, so exact formatting is not required.

Technical Footprint

Answers are salted-HMAC-checked field by field via `POST /api/exhibit-a`; nothing is compared in plaintext and nothing is revealed until every field is correct.

Answers

CITY [redacted]
STREET [redacted]
CAFÉ [redacted]
PHONE [redacted]
VIDEO [redacted]
STAGE 1 FLAG [redacted]

Redacted — select the bars above (or copy the surrounding text) to reveal.

STAGE 2 / 7

Exhibit B — EXIF Metadata Forensics

EVIDENCE: RECOVERED PHOTOGRAPH (recovered.jpg)

Scenario

A photo recovered from the scene is handed to the player intact, metadata and all. This stage teaches metadata literacy: the same file tells a different story depending on whether you only look at it, or actually inspect it.

Tasks

- Determine the GPS location encoded in the photo's EXIF data.
- Determine the file's true last-modified date.
- Determine the phone model used to take the photo.

How To Solve

Extract EXIF tags directly (any standard EXIF viewer/CLI tool works — `exiftool`, browser extensions, or manual hex inspection of the JPEG APP1 segment). The GPS tag resolves to a real, mappable coordinate; the modified date is written into three independent places (the image's own metadata tag, the file's on-disk modified time, and the server's `Last-Modified` HTTP header) so it reads identically no matter how the player inspects it; the camera model is a standard EXIF `Make`/`Model` tag.

Technical Footprint

The photo is served byte-for-byte from `data/evidence/recovered.jpg` — nothing about its metadata is altered in transit. Checked via `POST /api/exhibit-b`.

Answers

GPS LOCATION [redacted]
MODIFIED DATE [redacted]
PHONE MODEL [redacted]
STAGE 2 FLAG [redacted]

Redacted — select the bars above to reveal.

STAGE 3 / 7

Exhibit C — The Solstice PeopleHub Investigation

EVIDENCE: LIVE HR PLATFORM ACCESS

Scenario

The case file hands the player a portal access code and points them at **Solstice PeopleHub**, a full, live, internally-used HR platform running alongside the case file itself. This is the largest and most realistic stage: instead of a single puzzle, the player is asked to assess an entire application the way a real penetration tester would — nine independent vulnerabilities, of five different classes, hidden inside normal-looking business functionality across 23 pages and 12 backend route modules.

Stage Architecture

PeopleHub is reachable only once the outer game session has `stage3 = true` (`src/server.js` checks this before ever handing off). Past that gate it runs its own login (`/peoplehub/login.html` → `POST /api/peoplehub/auth/login`), its own session cookie (`phsid`, independent of the outer game's `sid`), and its own 9-role RBAC — enforced server-side in every route handler, never trusted to the frontend nav. Progress across the nine sub-tasks below is tracked in the `challenge_state` table, keyed by the *outer* session token so switching PeopleHub accounts never resets progress.

Tasks & How To Solve

3.1 — Login (SQL injection, authentication bypass)

The login form builds its query by raw string concatenation:

```
SELECT u.user_id, u.username, u.role_id, u.employee_id, u.is_active, r.name AS roleName
FROM users u JOIN roles r ON r.role_id = u.role_id
WHERE u.username = '${username}' AND u.password_hash = '${password}' LIMIT 1
```

Method: break out of the string with a quote and comment out the rest of the query so it always evaluates true. Because there is no `ORDER BY`, this blind bypass lands on whichever row SQLite naturally returns first (a low-privilege seat, by design). To escalate directly to the highest-privilege real account without knowing any username, use a `UNION SELECT` against the same `users`/ `roles` tables, ordered by role privilege, `LIMIT 1` — a correlated subquery pulls the real role name (not a placeholder), since the app authorizes by role *name*, not numeric ID.

BYPASS PAYLOAD [redacted]

ESCALATION PAYLOAD [redacted]

3.2 — Employee Directory (SQL injection, UNION, 13 columns)

The `search` parameter of `GET /api/peoplehub/employees` is concatenated; every dropdown filter beside it is properly parameterized. Requires the `directory` capability.

PAYLOAD [redacted]

3.3 — Employee Profile (SQL injection, UNION, 22 columns)

The `:id` path segment of `GET /api/peoplehub/employees/:id` is concatenated into the WHERE clause. The query selects every `employees` column (21) plus one joined department column.

PAYLOAD [redacted]

3.4 — People Analytics / Reports (SQL injection, UNION, 3 columns)

The `segment` filter of `GET /api/peoplehub/reports/:id/run` is concatenated. Result shape is small: `dimension, metric_name, metric_value`. Requires the `reports` capability.

PAYLOAD [redacted]

3.5 — HR Requests (SQL injection, UNION, 9 columns, no extra parenthesis)

Same pattern as the Directory, on `GET /api/peoplehub/hr-requests?search=` — but the query shape has no extra wrapping parenthesis to close, unlike Directory's.

PAYLOAD [redacted]

3.6 — Documents (Broken Object-Level Authorization / IDOR)

The document list view is correctly scoped — browsing normally never reveals anything wrong. The detail route (`GET /api/peoplehub/documents/:id`) checks classification-to-role access but never checks ownership for personal document types (contracts, compensation letters, performance reviews, leave approvals, onboarding/ offboarding, benefits, training records).

METHOD [redacted]

3.7 — Employee Data Import (XXE — External Entity Injection)

Administration → Employee Data Import (Administrator-only) runs a real DOCTYPE/ENTITY resolver, sandboxed to a synthetic filesystem on disk. A first decoy target proves the technique works without leaking anything real; the real chain requires following four files, each pointing to the next.

```
<?xml version="1.0"?>
<!DOCTYPE employees [<!ENTITY xxe SYSTEM "file:///challenge-data/<path>">]>
<employee><name>Test</name><role>Engineer</role><note>&xxe;</note></employee>
```

CHAIN STEP 1 [redacted]
CHAIN STEP 2 [redacted]
CHAIN STEP 3 [redacted]
CHAIN STEP 4 (PAYOFF) [redacted]

`reports/legacy-report.txt` is a dead-end decoy, mentioned nowhere in the real chain.

3.8 — HR Feedback (Stored Cross-Site Scripting)

Help & Support → "Share Feedback with HR" renders the review queue preview via unescaped `innerHTML`. Plain `<script>` tags do not execute this way; an event-handler attribute does.

PAYLOAD [redacted]

3.9 — Emergency Access (Business-Logic Bypass)

Security → Emergency Access has a normal Manager → Security → HR approval chain, plus an undocumented break-glass shortcut: a code, checked with `crypto.timingSafeEqual`, that skips straight to "HR Approved." The code is only discoverable via the XXE chain above (3.7) — nothing in the code enforces that dependency, but there is no other path to it.

EMERGENCY ACCESS CODE [redacted]

Tiered Hints Issued

Task	Tier 1	Tier 2	Tier 3
Login	The login form doesn't behave like a normal one under unusual input.	Classic SQL injection authentication bypass.	A quote breaks the query; comment out the rest. Look at what account you land on.
Escalation	There's a smarter way to pick who you become.	Same injection point can target a specific row.	`UNION SELECT` ordered by privilege, `LIMIT 1`.

Directory	The search box behaves oddly with certain characters.	UNION-based SQLi; count the columns.	Close the string, close the grouping, match columns, comment out the rest.
Employee Profile	The URL carries an identifier — what if it's not normal?	Same technique, different injection point, more columns.	Every employee column plus one joined column.
Reports	The segment filter may not validate the way you'd expect.	UNION injection, small result shape.	Three columns: dimension, metric name, metric value.
HR Requests	HR staff can search by more or less anything.	Same bug class as Directory — check the shape carefully.	Nine columns; no extra closing parenthesis needed.
Documents	Does every way of reaching a document go through the same check?	Authorization bug, not injection — IDOR.	The detail page checks document type, not ownership.
Import / XXE	The bulk import parses XML — what else can XML do?	XXE — external entity injection.	Start with the migration notes; follow what each file references next.
Feedback / XSS	A feedback "preview" is doing more than styling.	Stored XSS; the preview isn't escaping input.	`<script>` won't fire this way — an image with a broken source and an error handler will.
Emergency Access	There's a field that doesn't obviously connect to anything else.	Not a bug — a workflow that honors an old shortcut.	You've already found a code elsewhere in this investigation.

Stage 3 Completion

Back on the case file page, the player answers: *"What emergency access code was contained in the internal HR migration documentation?"* Combined with all nine sub-flags true, this completes the stage.

PORTAL ACCESS CODE (LOW-STAKES GATE) [redacted]

FINAL ANSWER [redacted]

STAGE 3 FLAG [redacted]

Seeded HR account passwords (helpdesk/admin/hr.manager/svc-directory) are randomly generated fresh on every reseed and are never meant to be known or guessed — access is via the SQL injection bypass above, not credential discovery.

STAGE 4 / 7

Exhibit D — XXE Injection & IDOR

EVIDENCE: LEAKED CONFIG + PAYSLIP SYSTEM

Scenario

A second, standalone lab (independent of PeopleHub) reusing the same real XXE resolver (`src/lib/xxe.js`) and a small IDOR-vulnerable payslip lookup. This stage runs against the outer game's own in-memory HR lab, not PeopleHub.

Tasks

- Submit an XML import that declares and references an external entity to leak a restricted internal config file, and recover the passphrase inside it.
- Exploit the payslip lookup's missing ownership check to view another employee's payslip and recover their outstanding debt figure.

How To Solve

The decoy target (`file:///etc/passwd`) confirms the XXE technique works without leaking anything real; the actual payoff is a leaked internal config file containing a backup passphrase. Separately, the player's own payslip ID is handed to them — trying nearby or different numeric IDs reveals other employees' records with no ownership check at all.

Technical Footprint

Checked via `POST /api/exhibit-d`; the passphrase field only accepts a correct answer if the session already recorded a successful XXE leak (`lab4.leaked`), so the debt figure alone can't be guessed into a solve.

Answers

LEAKED PASSPHRASE [redacted]

PAYSLIP OWED (ID 1007, "INCHAF", PAYROLL CLERK) [redacted]

STAGE 4 FLAG [redacted]

STAGE 5 / 7

Exhibit D½ — The Turn

NARRATIVE CHECKPOINT — NO PUZZLE

Scenario

A short narrative beat between the data-leak stages and the log-analysis stage: the case file reveals that the "outside" attacker is closer to home than the player assumed, recontextualizing everything found so far. There is no technical task here — the player acknowledges the beat via `POST /api/ack-dhalf` and proceeds directly to Stage 6.

Design Note

This stage exists purely for pacing — it prevents the log-analysis and disk-forensics stages from landing back-to-back with two data-leak stages without a narrative breath in between, and it is the moment the case reframes from "external breach" to "who inside Solstice Dynamics made this possible."

STAGE 6 / 7

Exhibit E — Apache Log Analysis

EVIDENCE: access.log + error.log

Scenario

The player is handed real Apache-style access and error logs covering the exact morning of the breach, mixed in with ordinary traffic from other visitors. The task is attacker attribution from raw logs — no application interaction, pure log analysis.

Tasks

- Identify the attacker's IP address from the pattern of requests (login attempt, SQLi probe in a search query, a feedback POST, an XML import POST, then five sequential payslip lookups for different IDs in under a minute).
- Recover a flag hidden inside one User-Agent string in the access log.

How To Solve

Cross-reference timestamps between the two logs: `error.log` flags the exact same source IP for a SQL-injection pattern, a UNION SELECT, a suspicious feedback submission, an XML external-entity declaration, and a payslip-probing pattern — all matching one IP across both files, distinguishing it from the normal background traffic. That IP's final request (a logout) carries an appended token in its User-Agent string.

Answers

ATTACKER IP [redacted]

USER-AGENT FLAG [redacted]

STAGE 6 FLAG [redacted]

STAGE 7 / 7

Exhibit F — Disk Forensics & Case Closed

EVIDENCE: thumbcache.txt + prefetch.txt

Scenario

The final stage: a recovered thumbnail cache index and a Windows Prefetch export from the attacker's own machine. This is classic post-compromise host forensics — proving what tool was used and when, from artifacts the attacker didn't clean up.

Tasks

- Identify the second-most-frequently-run program on the attacker's machine and its last-run timestamp.
- Recover a flag hidden in an orphaned thumbnail cache entry (a thumbnail with no matching file left on disk, but a surviving recovered EXIF comment).

How To Solve

Sort the Prefetch export by run count — the top entry is the OS shell itself (expected background noise); the second-highest is the tool of interest, together with its most recent run timestamp. Separately, one thumbnail cache row has no corresponding file on disk at all ("orphaned") — its recovered EXIF comment field is the hidden flag.

Case Closed

Solving Exhibit F reveals the final flag and unlocks the epilogue — the case file's closing narrative, confirming the full chain of events from Stage 1's OSINT trail through to the tool that ultimately breached Solstice Dynamics' HR platform.

Answers

PROGRAM [redacted]
LAST RUN [redacted]
THUMBNAIL CACHE FLAG [redacted]
STAGE 7 / FINAL FLAG [redacted]

Outer-game login (all stages)
This single credential pair gates entry to the entire case file, before Stage 1 even begins.

USERNAME [redacted]
PASSWORD [redacted]

Part III — Deployment Plan (Internet Hosting)

Critical finding — read before hosting this publicly

`fromLoopback()` in `src/server.js` gates `/ctf-admin.html` and `/api/ctf-admin/*` purely on whether the TCP connection reaching Node looks like `127.0.0.1`. The moment any reverse proxy or tunnel runs on the **same host** as the Node process — Caddy, nginx, Cloudflare Tunnel, all of them — every request Node receives arrives from that proxy's own local connection, so **every internet visitor's request looks like a loopback connection once it passes through the proxy**. Without an explicit block at the proxy layer, the developer-only reset/inspection panel becomes reachable by anyone on the internet, not just the machine's operator. §3.4 below shows the concrete fix — block the two admin paths in the proxy config, reach the panel over an SSH tunnel instead — and it is already applied in `deploy/Caddyfile`. Managed platforms (§3.5) don't have this failure mode, but trade it for the opposite one — see below.

3.1 Hosting Model — What "On The Internet" Means Here

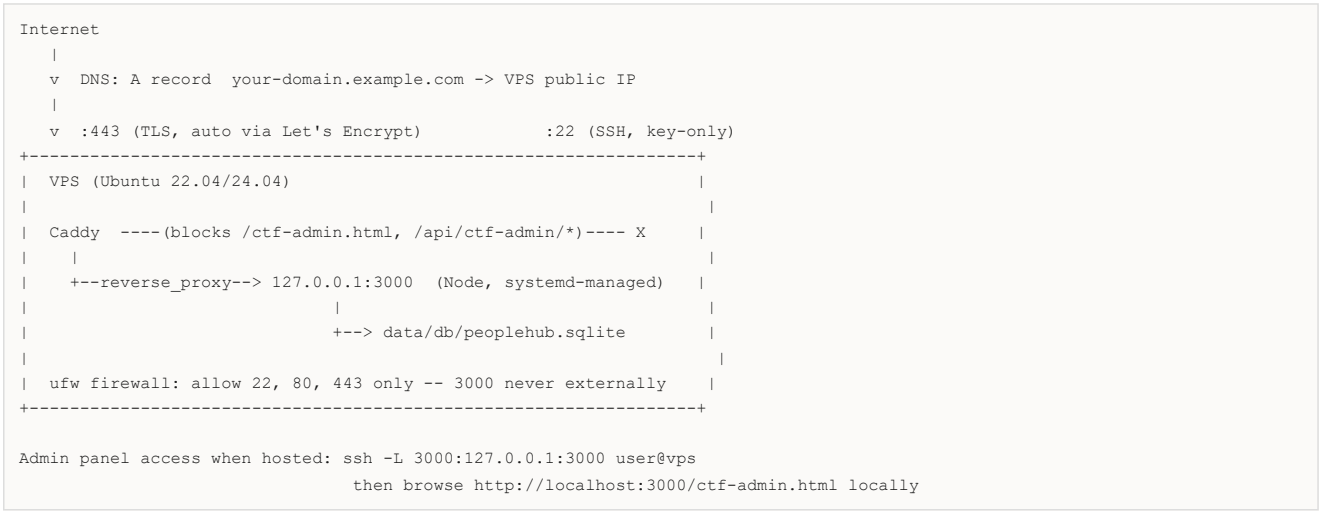
Two fundamentally different topologies are available, and they have opposite failure modes for the admin panel:

Topology	Admin panel behavior	Recommended for
A — VPS, proxy on the same host (Caddy/nginx/Cloudflare Tunnel running alongside Node on one machine)	Reachable by *everyone* unless explicitly blocked at the proxy (see the finding above) — because the proxy-to-Node hop always looks like loopback.	Full control, zero code changes needed beyond the proxy config. **Recommended.**
B — Managed platform (Render, Railway, Fly.io, ECS, Cloud Run — proxy terminates *off*-host, in the platform's own edge layer)	Unreachable by *anyone*, including the operator — the connection Node sees is never loopback.	Fastest to stand up, least ops; accept that resets go through the platform's shell/exec console instead of the web panel.

Not an option at all: serverless platforms (Vercel and similar) — see §3.6. They don't fit either row above because they don't run a persistent process in the first place.

This plan recommends **Topology A** as the primary path for full control, and covers **Topology B** concretely in §3.5 for anyone who wants less operational overhead and can live without the web-based admin panel.

3.2 Recommended Topology (Diagram) — VPS Path



3.3 VPS Provider Comparison & Sizing

Provider	Entry tier	Approx. cost	Notes
Hetzner Cloud	CX22 — 2 vCPU / 4 GB	~€4.50/mo	Best price/performance; EU-based datacenters.
DigitalOcean	Basic Droplet — 1 vCPU / 1 GB	$6/mo	Simplest UX, large community docs.
Linode (Akamai)	Nanode — 1 vCPU / 1 GB	$5/mo	Comparable to DO.
AWS Lightsail	512 MB–1 GB tier	$5/mo	Easiest on-ramp if you're already in AWS.

Prices are approximate and change over time — verify current rates before committing. The app is lightweight (in-memory session maps, one sqlite file, no background workers); 1 GB RAM handles a solo player or small cohort, 2 GB+ for a timed event with many simultaneous players. **My suggestion:** Hetzner CX22 or the DigitalOcean $6 droplet,

Ubuntu 24.04 LTS.

3.4 Step-By-Step VPS Deployment Runbook

1. **Provision** the VPS (Ubuntu 22.04/24.04). Create a non-root sudo user; disable password SSH auth, key-only.
2. **DNS:** point an A record (and AAAA if IPv6) at the VPS's public IP.
3. **Install Node 22.5+** (NodeSource repo or a version manager) — required for `node:sqlite`.
4. **Deploy the code** to `/opt/solstice-ctf`, then seed it: `npm run seed && npm run seed:peoplehub`.
5. **Install the systemd service** — `deploy/solstice-ctf.service` runs the app as a dedicated unprivileged `solstice-ctf` user, restarts automatically on crash, and starts on boot.
6. **Install Caddy** and deploy `deploy/Caddyfile` (edit the domain first) — automatic Let's Encrypt HTTPS, reverse-proxies to `127.0.0.1:3000`, and blocks the two admin paths (see §3.1's finding).
7. **Firewall:** `ufw allow OpenSSH; ufw allow 80; ufw allow 443; ufw enable`. Do not allow 3000 externally — Node binds all interfaces by default, so the firewall is the only thing stopping direct unproxied access to it.
8. **Verify:** `https://your-domain` loads the case file over TLS; confirm `https://your-domain/ctf-admin.html` returns 404 from the outside, then confirm the SSH-tunnel path (above) reaches it.

3.5 Managed Platform Hosting — Render, Railway, Fly.io

These platforms run your code as a normal, persistent, long-running process — unlike Vercel (§3.6), this shape is architecturally compatible with the app as-is, and needs the least manual server administration of any option in this plan.

Render (concrete steps)

1. Push this repository to GitHub/GitLab.
2. Render dashboard → **New** → **Web Service** (not "Static Site") → connect the repo.
3. **Build command:** leave blank or `echo "no build step"` — nothing is installed from npm.
4. **Start command:** `npm run render-start`, which runs `deploy/render-start.sh` — seeds only if `data/db/peoplehub.sqlite` doesn't already exist, otherwise starts directly against existing data. This means a redeploy never silently wipes player progress.
5. **Node version:** pinned via the `engines` field already in `package.json` (`>=22.5.0`, required for `node:sqlite`) — Render reads this automatically.
6. **Attach a persistent disk**, mounted at the service's working directory path for `data/`. Skip this and the database resets on every redeploy — Render's default filesystem is ephemeral between deploys.
7. Add a custom domain under the service's Settings if desired (Render issues the TLS cert automatically).

> **The admin panel will 404 for everyone, including you**
>
> Render's proxy terminates off-host (in Render's own edge layer, not on your service's container), so `fromLoopback()` never sees a real loopback connection — `/ctf-admin.html` and `/api/ctf-admin/*` fail closed rather than exposed. This is the safe failure mode (contrast with the VPS path, where you must remember to block those paths yourself), but it also means there's no way to reach the panel from a browser at all. Reset from Render's Shell tab instead: `node src/seed/seed.js` (full PeopleHub reseed) or `node src/seed/outer-seed.js` (outer game).

Railway and Fly.io follow the same shape: persistent process, a volume for `data/`, an off-host proxy with the same admin-panel trade-off, and their own equivalent of a start command / Node-version pin. The concrete dashboard steps differ but the architecture decision is identical to Render.

3.6 Why Not Vercel

> **Incompatible with this application's design — not a configuration issue**
>
> Vercel (and similar platforms built around the same model) runs code as short-lived **serverless functions** with an ephemeral, per-invocation filesystem — there is no single long-running process, and no guarantee two requests even hit the same underlying instance. This app depends on both of those things being false:
>
> - **Login sessions** (`sid`, `phsid`) live in plain in-memory `Map` objects inside one Node process (§1.5). On a serverless platform, the next request can hit a different, fresh invocation that has never seen that `Map` — the player would appear logged out immediately after logging in.
> - **PeopleHub's data** is a single SQLite file opened once and kept open (`src/lib/peoplehub/db.js`). Serverless filesystems are ephemeral and typically read-only outside a small `/tmp` scratch space that isn't shared across invocations — the database can't persist, and concurrent invocations can't safely share one file anyway.
> - **The admin reset panel** spawns a child process (`execFile` in `router.js`) — most serverless runtimes restrict or disallow this entirely.

None of this is fixable with configuration. It would require replacing the in-memory session Maps with an external store (Redis, etc.) and replacing the SQLite file with a hosted database — at which point it is no longer the same lightweight, dependency-free application this plan is for. If serverless is a hard requirement, treat that as a separate, larger project rather than a deployment option for the app as it exists today.

3.7 Environment & Secrets Handling

Variable	Purpose
`PORT`	HTTP port (default 3000) — Caddy/nginx/managed-platform proxy target this over loopback or the platform's internal networking.
`EVIDENCE_DIR`, `USERS_FILE`, `CHALLENGES_FILE`	Override outer-game data locations — useful for a throwaway test copy without touching real data.
`SEED_RNG_SEED`	Change the deterministic seed — treat as the one value worth keeping out of version control for a graded event, since it determines every answer.
`SEED_EMPLOYEE_COUNT` + 15 other `SEED_*` vars	Scale any PeopleHub data volume independently.

3.8 Reset / Reseed Operational Runbook

1. **VPS, between individual players:** SSH-tunnel to `/ctf-admin.html` (§3.2) and use "Reset Progress" — instant, no regeneration.
2. **VPS, between cohorts, same dataset:** "Reset Database" from the same tunneled panel, or directly on the box: `sudo -u solstice-ctf node src/seed/seed.js` (stop the service first: `systemctl stop solstice-ctf`).
3. **VPS, full reset:** `systemctl stop solstice-ctf && npm run seed && npm run seed:peoplehub && systemctl start solstice-ctf`.
4. **Render/Railway/Fly.io:** use the platform's shell/exec console — `node src/seed/seed.js` / `node src/seed/outer-seed.js` — since the web panel is unreachable (§3.5).

3.9 Backup & Recovery

`data/` is the entire stateful footprint — one sqlite file plus a handful of small JSON/text files. On a VPS, a daily cron snapshot is enough:

```
0 3 * * * tar czf /var/backups/solstice-ctf-$(date +\%F).tar.gz -C /opt/solstice-ctf data
```

On a managed platform, use its own volume-snapshot feature if offered. Recovery is just restoring the archive/snapshot back over `data/` and restarting. Because seeding is deterministic, a backup is only needed to preserve *live player progress* — the base dataset itself can always be regenerated identically from the same `SEED_RNG_SEED`.

3.10 Security Hardening Checklist

- **VPS:** admin paths blocked at the proxy layer (§3.1 finding) — verify this before anything else.
- SSH: key-only auth, root login disabled, consider `fail2ban` for brute-force protection on port 22.
- Firewall: only 22/80/443 open; 3000 never exposed externally.
- App-level rate limiting is already in place (login attempts, answer submissions, lab requests — all throttled per-IP/per-session in `src/server.js`) and becomes accurate once the proxy forwards real client IPs via `CF-Connecting-IP`.
- Session cookies are already `HttpOnly; SameSite=Strict`; the `Secure` flag is added automatically once `X-Forwarded-Proto: https` is forwarded (the provided Caddyfile does this; most managed platforms do this by default).
- Rotate `SEED_RNG_SEED` between separately-graded cohorts so answers aren't shared across runs.
- On a VPS, the systemd unit runs as an unprivileged dedicated user with `ProtectSystem=strict` — keep it that way rather than running the app as root.

Appendix — Reference

A.1 Environment Variable Reference

See §3.7 above — full table repeated here for quick lookup during a live deployment.

Variable	Default
PORT	3000
SEED_RNG_SEED	20260101
SEED_EMPLOYEE_COUNT	1000
SEED_AUDIT_COUNT	2100
SEED_DOCUMENT_COUNT / SEED_REQUEST_COUNT	520 / 520
SEED_TRAINING_ENROLLMENT_COUNT	1050

A.2 File Map Quick Reference

To change…	Edit…
Outer-game answers/flags	`src/seed/outer-seed.js`
PeopleHub data volume	`src/seed/config.js` or `SEED_*` env vars
A PeopleHub role's permissions	`src/lib/peoplehub/rbac.js`
A PeopleHub business page's backend	`src/lib/peoplehub/routes/*.js`
A PeopleHub business page's frontend	`public/peoplehub/*.html` + `*.js`
The DB schema	`data/db/schema.sql`
The XXE sandbox filesystem	`data/challenge-data/`
VPS reverse proxy / systemd config	`deploy/Caddyfile`, `deploy/solstice-ctf.service`
Render start behavior	`deploy/render-start.sh`

— End of Case File —
SOLSTICE DYNAMICS LTD. — INTERNAL SECURITY REVIEW — TRAINING USE ONLY